

Digital Nurture 5.0 DeepSkilling

Test driven development and Logging framework

Tasks:

1. Setting Up JUnit

Pom.xml

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>JUnitDemo</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>JUnitDemo</name>

  <dependencies>

    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>

  </dependencies>

</project>
```

Calculator.java

```
package com.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

}
```

CalculatorTest.java

```
package com.example;

import static org.junit.Assert.assertEquals;

import org.junit.Test;

public class CalculatorTest {

    @Test
    public void testAddition() {

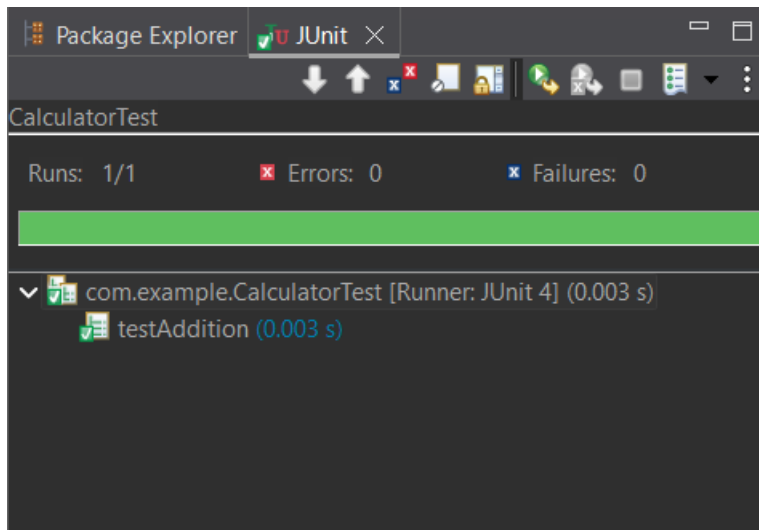
        Calculator calculator = new Calculator();

        int result = calculator.add(10, 20);

        assertEquals(30, result);
    }

}
```

Output



2. Assertions in JUnit

AssertionsTest.java

```
package com.example;

import static org.junit.Assert.*;
import org.junit.Test;

public class AssertionsTest {

    @Test
    public void testAssertions() {

        // Assert Equals
        assertEquals(5, 2 + 3);

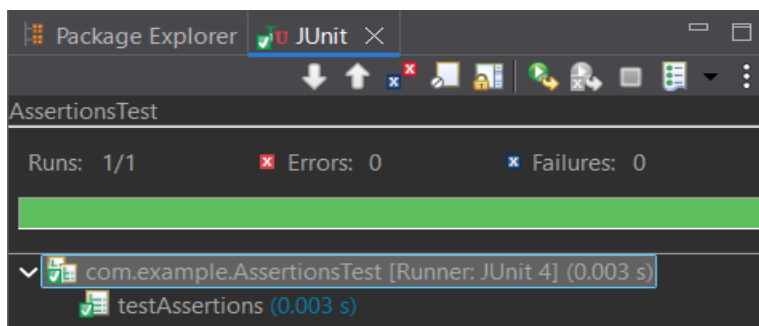
        // Assert True
        assertTrue(5 > 3);

        // Assert False
        assertFalse(5 < 3);

        // Assert Null
        assertNull(null);

        // Assert Not Null
        assertNotNull(new Object());
    }
}
```

Output



3. Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

```
package com.example;

import static org.junit.Assert.assertEquals;

import org.junit.After;
import org.junit.Before;
import org.junit.Test;

public class CalculatorAAATest {

    private Calculator calculator;

    @Before
    public void setUp() {
        System.out.println("Setting up Calculator...");
        calculator = new Calculator();
    }

    @After
    public void tearDown() {
        System.out.println("Cleaning up...");
        calculator = null;
    }

    @Test
    public void testAddition() {

        // Arrange
        int num1 = 10;
        int num2 = 20;

        // Act
        int result = calculator.add(num1, num2);

        // Assert
        assertEquals(30, result);
    }
}
```

Output

```
Finished after 0.024 seconds

Runs: 1/1      ✖ Errors: 0      ✖ Failures: 0

> com.example.CalculatorAAATest [Runner: JUnit 4] (0.001 s)
```

```
Setting up Calculator...
Cleaning up...
```

4. Mocking and Stubbing

ExternalAPI.java

```
package com.example;

public interface ExternalApi {

    String getData();

}
```

MyService.java

```
package com.example;

public class MyService {

    private ExternalApi externalApi;

    public MyService(ExternalApi externalApi) {
        this.externalApi = externalApi;
    }

    public String fetchData() {
        return externalApi.getData();
    }

}
```

MyServiceTest.java

```
package com.example;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.when;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceTest {

    @Test
    public void testExternalApi() {

        // Create Mock Object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Stub Method
        when(mockApi.getData()).thenReturn("Mock Data");

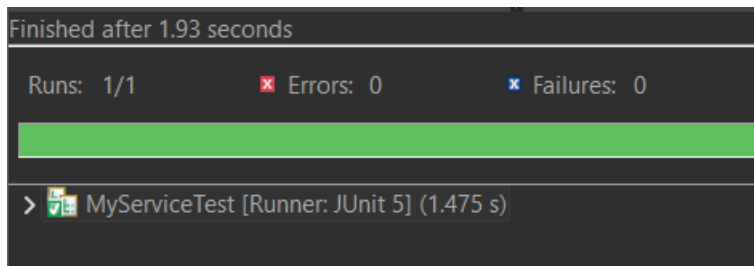
        // Inject Mock into Service
        MyService service = new MyService(mockApi);

        // Call Method
        String result = service.fetchData();

        // Verify Result
        assertEquals("Mock Data", result);
    }

}
```

Output



5. Verifying Interactions

MyServiceVerifyTest.java

```
package com.example;

import static org.mockito.Mockito.verify;

import org.junit.jupiter.api.Test;
import org.mockito.Mockito;

public class MyServiceVerifyTest {

    @Test
    public void testVerifyInteraction() {

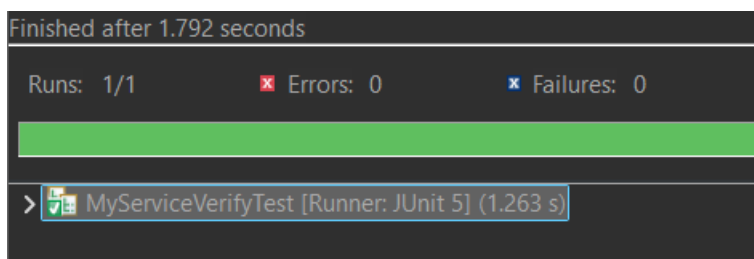
        // Create Mock Object
        ExternalApi mockApi = Mockito.mock(ExternalApi.class);

        // Create Service
        MyService service = new MyService(mockApi);

        // Call Method
        service.fetchData();

        // Verify Interaction
        verify(mockApi).getData();
    }
}
```

Output



6. Logging Error Messages and Warning Levels

LoggingExample.java

```
package com.example;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

public class LoggingExample {

    // Create Logger Object
    private static final Logger logger =
        LoggerFactory.getLogger(LoggingExample.class);

    public static void main(String[] args) {

        logger.error("This is an error message");

        logger.warn("This is a warning message");

    }

}
```

Output

```
11:58:04.629 [main] ERROR com.example.LoggingExample - This is an error message
11:58:04.632 [main] WARN com.example.LoggingExample - This is a warning message
```